

Algoritmos e Programação I

Prof. Dr. Amaury Antônio de
Castro Junior

Módulo 5 - Estratégias de apoio à programação

Unidade 2 - Noções de testes automatizados

O que já estudamos?

- Tipos simples (números, caracteres) e comandos básicos de manipulação de dados em Python;
- Estruturas de controle de fluxo (condicionais e laços);
- Strings e tipos de dados compostos (vetores e matrizes);
- Fundamentos e benefícios da modularização;
- Importância e técnicas de documentação e depuração de código.

Conceito e importância dos testes

- Testes automatizados são códigos que testam outro código;
- **Humanos erram**: à medida que programas crescem, surgem mais *bugs*. Testes evitam que esses erros cheguem ao usuário.

Conceito e importância dos testes

- *Bugs* podem só causar incômodo (*apps* que travam) ou, em casos extremos, prejuízo financeiro e até perda de vidas (ex.: falhas em *software* de foguetes ou usinas);
- **Objetivo**: desenvolver *softwares* de alta qualidade com o mínimo possível de erros.

Tipos de erros

- **Erros de sintaxe**: fáceis de detectar (compilador/interpretador mostra);
- **Erros de lógica**: mais difíceis de encontrar, pois o programa roda, mas se comporta errado só em alguns casos:
 - Esses são, principalmente, os que queremos perceber com testes.

Testes manuais x testes automatizados

- Testes manuais:
 - Testadores clicando e repetindo cenários;
 - Cansativos, fáceis de pular e pouco escaláveis.
- Testes automatizados:
 - Permitem rodar vários casos de teste sempre que o código muda;
 - Aumentam a confiabilidade.

Ferramenta sugerida: pytest (Python)

- Disponível em: <https://docs.pytest.org>
- Instalação típica: `pip install pytest` (veja docs em [pytest.org](https://docs.pytest.org)).
- Convenções importantes:
 - Arquivos de teste: nome começam com `test_` (ex.: `test_algo.py`).
 - Funções de teste: começam com `test_` (ex.: `def test_soma() :`).
 - `pytest` procura automaticamente esses arquivos e funções.
- Para rodar: executar `pytest nome_do_arquivo.py` (na pasta onde está salvo o arquivo).
- Saída: testes que passam aparecem em verde; os que falham, em vermelho. `pytest` mostra exatamente em que linha/falha ocorreu.

Exemplo prático 1 - vamos treinar

```
# conteúdo do arquivo test_sample.py
# definição da função inc - soma um ao valor de x
# e retorna o resultado
def inc(x):
    return x + 1

# definição da função de teste - confere se o valor
# é correto
def test_answer():
    assert inc(3) == 5
```

Exemplo prático 1 - vamos treinar

```
# conteúdo do arquivo test_sample.py
# definição da função inc - soma um ao valor de x
# e retorna o resultado
def inc(x):
    return x + 1

# definição da função de teste - confere se o valor
# é correto
def test_answer():
    assert inc(3) == 5
```

Exemplo prático 1 - vamos treinar

```
# conteúdo do arquivo test_sample.py
# definição da função inc
# e retorna o resultado
def inc(x):
    return x + 1

# definição da função de teste
# que verifica se o resultado
# é correto
def test_answer():
    assert inc(3) == 5
```

Resultado:

O teste falha (porque o correto é 4). O teste apontou exatamente o problema. No exemplo, o próprio teste estava errado. Ajustando o teste para `assert inc(3) == 4`, o pytest passará.

Exemplo prático 2 - soma de números

- Problema: somar todos os números de 1 até **n** (incluindo **n**), em que o valor de **n** é fornecido pelo usuário.
- Implementação típica (exemplo em pseudocódigo):
- Corpo da função em Pseudocódigo:
- Corpo da função em Python:

```
total = 0
para i de 1 até n:
    total = total + i
retornar total
```

```
total = 0
for i in range(n):
    total = total + i
return total
```

Arquivo test_soma.py

```
def somatorio(n):  
    total = 0  
    for i in range(n):  
        total = total + i  
    return total  
  
def test_soma0():  
    assert somatorio(0) == 0  
  
def test_soma3():  
    assert somatorio(3) == 6
```

```
def test_soma_negativo():  
    assert somatorio(-10) == -1  
  
def test_soma8():  
    assert somatorio(8) == 36  
  
def test_soma10():  
    assert somatorio(10) == 55
```

Análise da execução do teste 2

- Se a implementação em Python usou um intervalo que não inclui **n** (por exemplo `range(n)` em vez de `range(1, n+1)`), haverá um erro de uma unidade (*off-by-one*).
 - Por exemplo: `soma(4)` devolvendo 6 indica esse problema; corrigir o laço para incluir **n** resolve o erro.
- Um bom conjunto de testes simples detecta erros comuns (*off-by-one*, tratamento de zero/negativos) e ajuda a garantir que a função se comporte conforme esperado para diferentes entradas.

Boas práticas ensinadas nessa aula

- Sempre pense nos casos nos quais o programa pode falhar (valores limite, negativos, entradas inválidas);
- Escreva testes automatizados para cada caso relevante;
- Faça os testes com frequência durante o desenvolvimento do *software*.

Boas práticas ensinadas nessa aula

- Mantenha um conjunto de testes que cubra o máximo possível do código (objetivo inicial: todas as linhas; depois, caminhos mais completos);
- Use ferramentas (pytest) para facilitar a escrita e a execução dos testes e para integrar com o ciclo de desenvolvimento.

Dicas finais e recomendações

- **Automatizar é o caminho**: substituir volumes de testes manuais por automáticos vale a pena;
- Leia a documentação do pytest para se aprofundar: há muitos recursos, inclusive avançados.

Dicas finais e recomendações

- Testar não é uma só garantia técnica, mas também demonstra cuidado com o usuário e com o futuro do produto (evita perda de clientes, reputação, prejuízos).
- É isso: **testar, testar e testar!**
- Escrever testes é tão importante quanto escrever o programa.

Referências

BANIN, S. L. **Python 3: conceitos e aplicações** - uma abordagem didática. São Paulo: Erica, 2018. ISBN 9788536530253.

MANZANO, J. A. N. G. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29 ed. São Paulo: Erica, 2019. ISBN 9788536531472.

PERKOVIC, L. **Introdução à computação usando Python: um foco no desenvolvimento de aplicações**. Rio de Janeiro: LTC, 2016. ISBN 9788521630937.

WENTWORTH, P.; ELKNER, J.; DOWNEY, A. B.; MEYERS, C. **How to think like a computer scientist: learning with Python 3**. [S.L.: S. N.], 2012. Disponível em: <https://link.ufms.br/Vow0N>. Acesso em 05 mai. 2025.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

